豆瓣电影短评舆情分析平台项目报告

1. 封面信息

- 项目名称：豆瓣电影短评舆情分析平台
- 项目类型：基于 Web Scraping 的数据服务型 Web 应用
- 技术方向：数据抓取、文本清洗、情感分析、可视化展示
- 作者信息：邓睿涵 / 210110067 / 奥德赛
- 提交日期：2026-04-13
- GitHub 仓库链接：https://github.com/dengruihan/Douban_Film_Comment-monitoring
- 在线部署链接：待补充
- 演示视频链接：待补充

2. 项目摘要

本项目围绕“如何快速理解一部电影的公开舆情”这一实际问题，开发了一个基于 Python 和 Streamlit 的电影短评舆情分析平台。用户输入电影名称后，系统会自动搜索目标电影、抓取豆瓣移动端公开评论数据、进行文本清洗与情感分析，并通过图表、词云和表格展示结果。

项目实现了从“外部数据获取”到“分析结果展示”的完整流程，具有明确的数据来源、可复现的技术方案和可视化界面，符合课程对“以网络数据为基础构建实际服务”的要求。

3. 项目背景与问题定义

在观影决策、影片评价研究和舆情观察中，用户通常需要阅读大量短评来判断电影口碑。但直接浏览原始评论存在以下问题：

- 评论数量大，人工筛选成本高；
- 原始页面信息密集，难以快速提炼整体倾向；
- 评论文本存在噪声，直接使用不利于统计分析；
- 单纯展示原始数据不能体现“数据服务”价值。

因此，本项目的目标是构建一个面向普通用户的舆情分析平台，通过抓取电影公开短评并进行加工处理，为用户提供：

- 电影整体情感分布；
- 评论热度随时间的变化趋势；
- 高频关键词与核心讨论内容；

- 不同电影之间的同类对比能力。

4. 数据来源与采集方法

4.1 数据来源

本项目使用豆瓣移动端公开接口作为数据来源，主要包括：

- 搜索接口：`m.douban.com/rexxar/api/v2/search/movie`
- 电影详情接口：`m.douban.com/rexxar/api/v2/movie/{id}`
- 评论接口：`m.douban.com/rexxar/api/v2/movie/{id}/interests`

抓取字段主要包括：

- 电影信息：电影 ID、电影名称、平均评分、分类信息
- 评论信息：用户名、星级、评论文本、点赞数、评论时间

4.2 采集流程

1. 用户输入电影名称。
2. 系统调用搜索接口获取候选电影列表。
3. 用户选择目标电影后，系统读取电影 ID。
4. 系统抓取电影详情与评论数据。
5. 数据写入本地 SQLite 数据库。
6. 前端读取数据库并完成分析与展示。

4.3 数据采集策略

为避免对目标网站造成过大压力并降低反爬风险，项目采用了以下策略：

- 默认限制采集页数为 10 页；
- 每页固定采集 20 条评论；
- 在请求之间加入随机延迟；
- 优先使用移动端接口，减少网页结构变化带来的不稳定性；
- 对异常请求和失败情况增加重试机制。

5. 合规、伦理与安全说明

5.1 合规性考虑

本项目仅用于课程学习与技术演示，数据来源为公开可访问内容。系统仅在用户主动发起查询时抓取单部电影数据，不进行大规模批量抓取。

当前版本已经实现：

- 请求频率控制；
- 可选 Cookie 环境变量配置；

- 不在代码中强制依赖登录态；
- 敏感配置通过环境变量管理。

后续若面向公开部署，应进一步补充：

- robots.txt 检查与访问许可确认；
- 更严格的速率限制；
- 访问日志与异常告警；
- 对第三方平台服务条款的人工确认。

5.2 伦理说明

- 仅采集公开评论字段，不采集隐私敏感信息；
- 不对用户个体建立画像；
- 结果展示以聚合分析为主，不将评论用于商业化二次利用；
- 本系统的分析结果仅作辅助参考，不等同于平台官方评价结论。

5.3 安全说明

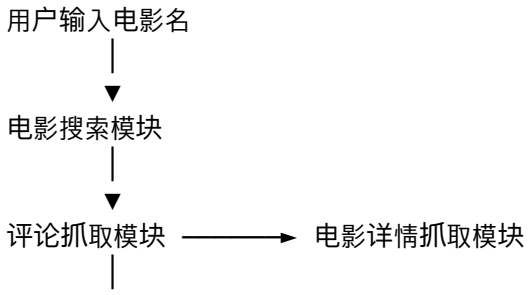
- API Key、Cookie 等敏感信息通过环境变量注入；
- 仓库中不应保留真实密钥；
- SQLite 数据仅用于本地或课程演示环境；
- 公开部署时应移除所有本地测试凭据和临时调试配置。

6. 技术方案与系统架构

6.1 技术栈

- 前端与展示：Streamlit
- 数据抓取：requests
- 解析：BeautifulSoup、lxml
- 数据处理：pandas
- 中文分词：jieba
- 情感分析：SnowNLP
- 图表可视化：pyecharts、streamlit-echarts、matplotlib、wordcloud
- 数据库存储：SQLite
- 可选 AI 能力：DeepSeek API（用于关键词筛选）

6.2 系统架构



```
        ▼
SQLite 数据库存储
        │
        ▼
数据清洗与情感分析模块
        │
        ▼
可视化展示模块（饼图 / 词云 / 时间轴 / 对比表）
```

6.3 模块设计

- `app.py`
  - 负责页面交互、用户输入、结果展示和进度反馈。
- `crawler/douban_spider.py`
  - 负责搜索电影、获取电影详情、抓取评论。
- `analysis/data_cleaner.py`
  - 负责清洗 HTML、链接、特殊字符和空白符。
- `analysis/nlp_processor.py`
  - 负责情感分类、词频统计、热度时间序列、评论权重计算。
- `database/db_manager.py`
  - 负责初始化数据库、去重插入、读取电影信息和评论记录。

7. 核心实现逻辑

7.1 情感分析逻辑

项目采用“星级优先 + 文本补充”的策略：

- 1-2 星：直接视为负面
- 5 星：直接视为正面
- 3-4 星：结合文本情感分数进一步判断
- 无星级短文本：默认视为中性

这种方式比单纯依赖情感词典更适合电影短评场景，因为星级本身已经体现了用户的明确评价倾向。

7.2 评论权重设计

为避免普通评论与高价值评论被同等看待，系统引入评论权重：

- 星级越极端，权重越高；
- 点赞数越高，权重越高；
- 情感强度越高，权重越高。

最终可同时展示普通好评率和加权好评率，从而提升分析结果的解释力。

7.3 关键词筛选逻辑

项目支持两种关键词筛选方式：

- AI 智能筛选：优先保留与电影主题更相关的高频词；
- 正则降级筛选：在 AI 不可用时，基于停用词、正则规则和词性进行筛选。

8. 系统实现结果

8.1 页面功能说明

系统当前实现的主要功能包括：

- 电影名称搜索与候选电影选择；
- 评论抓取进度展示；
- 电影评分与评论总数展示；
- 情感分布图；
- 高频词云；
- 评论热度时间轴；
- 评论详情排序展示；
- 同类电影情感对比。

8.2 运行截图与结果说明

图 1 首页输入界面

图1 首页输入界面

图 1 展示了系统首页的电影名称输入区域。用户无需手动查询电影 ID，只需输入电影名称即可开始搜索，降低了使用门槛。

图 2 搜索结果下拉选择界面

图2 搜索结果下拉选择界面

图 2 展示了系统根据用户输入返回候选电影列表后的下拉选择界面。用户可直接从结果中选择目标影片，系统会自动绑定对应电影 ID 进入后续分析流程。

图 3 评论抓取进度界面

图3 评论抓取进度界面

图 3 展示了评论采集时的进度反馈页面。系统会实时显示当前抓取页数、当前页评论数量和累计评论数，从而证明数据并非静态展示，而是实际进行了外部数据获取与更新。

图 4 电影基础信息与核心指标展示（一）

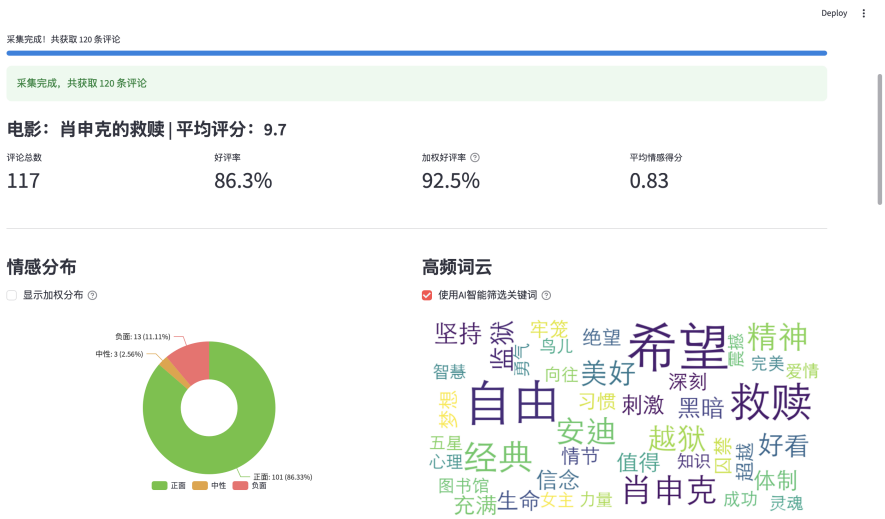


图4 电影基础信息与核心指标展示一

图 4 展示了系统对电影基础信息和核心指标的展示结果，包括电影名称、平均评分、评论总数、好评率、加权好评率和平均情感得分。这部分体现了系统对原始评论数据进行了统计与增值处理，而不是简单显示原始文本。

图 5 电影基础信息与核心指标展示（二）

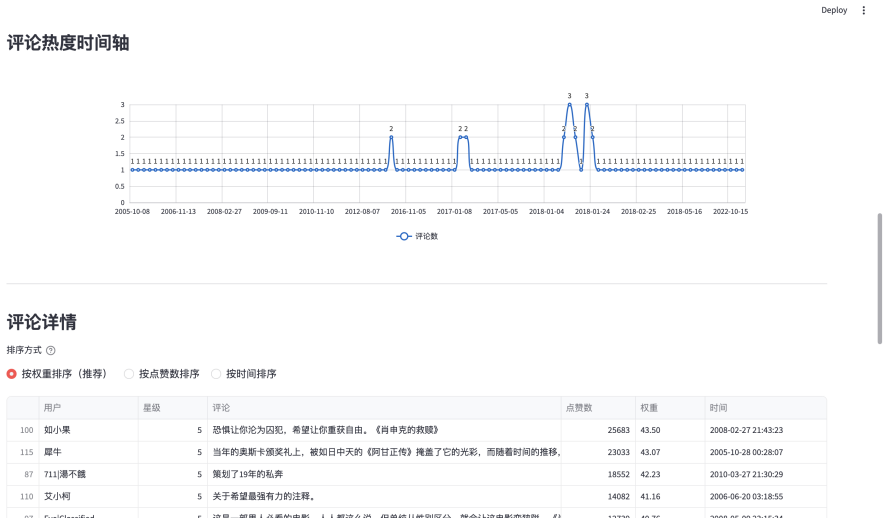


图5 电影基础信息与核心指标展示二

图 5 展示了分析页面的中部结果区域，可用于体现情感分布图、关键词可视化或其他中间分析结果。该部分展示了系统将评论内容进一步转换为更适合用户理解的图形信息。

图 6 电影基础信息与核心指标展示（三）

	名称	豆瓣评分	好评率	加权好评率	评论数
0	肖申克的救赎	9.7	86.3	92.5	117
1	奇迹男孩	8.6	89.2	93	120
2	你好，李焕英	7.7	80.7	88.6	119
3	哪吒之魔童闹海	8.4	64.7	66.4	116
4	我是哪吒2之英雄归来	2.6	1.7	0.4	118

图6 电影基础信息与核心指标展示三

图 6 展示了分析页面的后续结果区域，可用于体现评论热度时间轴、评论明细表或同类电影对比等内容。通过这部分，系统进一步展示了评论数据在时间维度和对比维度上的分析价值。

8.3 截图小结

以上截图覆盖了本系统的主要使用流程：

- 用户输入电影名称；
- 系统返回候选影片供选择；
- 系统抓取并更新评论数据；
- 系统展示统计指标与分析结果。

这些运行结果说明，本项目不仅能够抓取外部数据，还能对数据进行清洗、分析与可视化处理，满足课程作业中“提供增值服务”的要求。

9. 测试与验证

项目包含多个测试脚本，覆盖以下内容：

- 配置模块测试
- 数据库模块测试
- 爬虫解析测试
- 数据清洗测试
- NLP 情感分析测试
- 修复验证与回归测试

已完成的测试材料包括：

- `docs/回归测试报告.md`
- `docs/测试报告_二号测试员.md`

- `tests/comprehensive_test.py`

这些测试结果表明，系统在核心功能上具备较好的稳定性和可运行性。

10. 部署与可访问性

当前项目基于 Streamlit 构建，适合部署到以下平台：

- Streamlit Community Cloud
- Render
- Railway

部署要求包括：

- 安装 `requirements.txt` 中的依赖；
- 配置环境变量；
- 保证应用入口为 `app.py`；
- 保证页面在不同窗口尺寸下可正常查看。

正式提交时，应在本节填入实际可访问的公开部署链接。

11. 项目价值、局限性与改进方向

11.1 项目价值

- 将分散评论整理为结构化信息；
- 帮助用户快速理解电影口碑；
- 体现了从抓取到分析再到展示的完整工程链路；
- 具有进一步扩展到书籍、电视剧、商品评论分析的潜力。

11.2 局限性

- 当前数据源主要依赖单一平台；
- 情感分析仍以轻量模型与规则为主，复杂语义处理有限；
- 没有实现自动化 robots 检查；
- 公开部署时还需要更严格的访问频率和异常控制。

11.3 后续改进方向

- 支持更多数据源与多平台评论对比；
- 使用更强的中文情感模型提升准确率；
- 增加缓存和异步抓取以提升性能；
- 加入更完善的部署配置和后台监控；
- 增加导出报告和自定义筛选功能。

12. 结论

本项目完成了一个面向真实问题的 Web 数据服务应用，能够从公开网络数据中提取有价值的信息，并以结构化、可视化形式反馈给用户。项目具备明确的问题背景、可执行的技术路线和较完整的功能实现，符合课程中对“抓取外部数据并提供增值服务”的要求。

后续如能补充真实部署链接、演示视频和运行截图，将更适合作为完整课程结项作品提交。